PROGRAMAÇÃO I

# ESTRUTURAS DE DADOS EM PYTHON

VOLUME II • CADERNO 02

## TUPLAS, DICIONÁRIOS E CONJUNTOS

Três formas diferentes de guardar informações: por posição, por chave e sem duplicação.

**COMO ESTUDAR** Comece pelo problema concreto, acompanhe o estado dos dados antes e depois de cada operação e só então leia o código. Execute os exemplos na ordem. Quando aparecer um termo técnico, ele será definido no próprio capítulo. Não é necessário decorar comandos antes de entender o comportamento.

**CONTEXTO DIDÁTICO** Os exemplos acompanham uma rede fictícia de lojas, com produtos, clientes, estoque, pedidos e atendimento. Os registros são exemplos de aula, não dados comerciais reais. Este caderno pressupõe os fundamentos de Python e o Caderno 01 (listas).

Série de cadernos independentes e complementares • Texto explicativo, exemplos executáveis e roteiros físicos de representação.

## Mapa de aprendizagem deste caderno

Ao final, o estudante conseguirá distinguir sequências imutáveis, cadastros formados por chave/valor e conjuntos de elementos únicos; além de escolher uma estrutura para cada necessidade concreta.

As representações em tabelas mostram o que o computador armazena; as demonstrações físicas mostram como uma pessoa poderia simular as mesmas operações com cartões ou colegas em posições fixas. **O número da posição não se move: o elemento é que muda.**

## 1. Uma sequência que não deve mudar: tuplas

Na expedição de uma loja, a identificação de um pacote pode ser descrita por um conjunto pequeno de informações: código do pedido, nome da unidade e data de envio. Podemos guardar esses valores em posições fixas. A estrutura chamada **tupla** preserva a sequência e não permite substituir diretamente seus elementos após a criação.

Código 1 — ler informações de uma tupla.

```
remessa = (501, "Centro", "2026-11-05")
print(remessa[0])
print(remessa[-1])
print(len(remessa))
```

A saída é 501, depois 2026-11-05, depois 3. Assim como nas listas, os índices começam em zero; índices negativos contam da direita para a esquerda. A diferença está no que acontece quando queremos trocar um elemento.

| Índice | 0 | 1 | 2 |
|---|---|---|---|
| Valor | 501 | Centro | 2026-11-05 |

**REPRESENTAÇÃO FÍSICA** Três alunos ocupam posições numeradas 0, 1 e 2, com etiquetas de identificação de uma remessa. Na simulação de uma tupla, a equipe pode consultar qualquer posição, mas não pode substituir isoladamente a etiqueta: deve representar um novo registro inteiro.

```
remessa = (501, "Centro", "2026-11-05")
# Esta linha provocaria TypeError:
# remessa[1] = "Eldorado"
remessa_corrigida = (501, "Eldorado", "2026-11-05")
```

A variável remessa_corrigida referencia **outra tupla**. Isso não significa que o valor anterior foi editado. A tupla original continua com Centro. A imutabilidade protege a composição da sequência, não transforma um registro em verdade absoluta: informações da vida real podem mudar, mas precisam ser representadas por uma nova tupla ou por outra estrutura apropriada.

## 2. Operações com tuplas: leitura e desempacotamento

As tuplas podem ser percorridas, contadas e consultadas por posição. O fato de serem imutáveis não impede ler, comparar ou calcular. Muitas operações que consultam sequências funcionam igualmente em listas e tuplas.

```
precos = (24.90, 18.50, 24.90, 59.90)
print(precos.count(24.90))  # 2 ocorrências
print(precos.index(59.90))  # índice 3
print(18.50 in precos)     # True
for preco in precos:
    print(f"R$ {preco:.2f}")
```

### Desempacotamento: dar nome a cada posição

Quando sabemos que um registro possui exatamente três componentes, podemos recebê-los em três variáveis ao mesmo tempo. Esse procedimento chama-se **desempacotamento**. É semelhante a retirar os valores de três caixas e colocar etiquetas legíveis em cada uma.

```
produto = (101, "Shampoo Nutritivo", 24.90)
codigo, nome, preco = produto
print(nome)
print(preco)
```

A atribuição exige que a quantidade de destinos corresponda à quantidade de elementos, a menos que usemos uma variável com asterisco para receber parte da sequência. Para uma tupla de um único elemento, a vírgula é indispensável: `(10,)` é tupla; `(10)` é apenas o número 10 entre parênteses.

```
um_codigo = (101,)
print(type(um_codigo))  # <class 'tuple'>
```

**CUIDADO CONCEITUAL** Uma tupla que contém uma lista é imutável no sentido de que a posição da lista dentro da tupla não pode ser substituída. Mas o conteúdo da lista interna pode mudar. Por isso, “tupla imutável” não significa “tudo o que está dentro dela é inalterável”.

## 3. Quando escolher lista ou tupla?

O critério de escolha não é o tamanho: ambas podem armazenar poucos ou muitos elementos. O critério principal é a intenção de uso. Uma lista é apropriada quando precisamos inserir, remover ou reorganizar itens; uma tupla é apropriada quando tratamos uma coleção posicional como registro que não deve ser alterado em partes.

| Necessidade da loja | Estrutura indicada | Justificativa |
|---|---|---|
| Carrinho de compras em edição | Lista | Produtos entram e saem durante a compra. |
| Coordenadas de uma entrega (latitude, longitude) | Tupla | Duas posições com significados estáveis. |
| Sequência de produtos da vitrine | Lista | A vitrine precisa ser reordenada. |
| Identificação temporária (código, unidade) | Tupla | Registro pequeno, lido em conjunto. |

Em Python, funções frequentemente devolvem mais de uma informação por meio de uma tupla, mesmo quando os parênteses não aparecem na instrução `return`.

```
def situacao_estoque(quantidade, minimo):
    precisa_repor = quantidade < minimo
    return quantidade, precisa_repor

atual, reposicao = situacao_estoque(7, 10)
print(atual, reposicao)  # 7 True
```

## 4. Um cadastro por nomes de campos: dicionários

Imagine consultar um produto representado somente por posições: índice 0 é o código, índice 1 o nome e índice 2 o preço. Essa representação pode funcionar, mas obriga o programador a

memorizar o significado de cada número. Um **dicionário** associa **chaves** a **valores**, deixando cada campo identificável pelo seu nome.

```
produto = {
    "codigo": 101,
    "nome": "Shampoo Nutritivo",
    "preco": 24.90,
    "estoque": 18
}
print(produto["nome"])
print(produto["estoque"])
```

| Chave | Valor armazenado |
|---|---|
| codigo | 101 |
| nome | Shampoo Nutritivo |
| preco | 24.90 |
| estoque | 18 |

Observe: a chave `"estoque"` não é um índice numérico. Ela identifica o campo desejado. Em um dicionário, as chaves são únicas; atribuir outro valor à mesma chave **atualiza** a entrada, em vez de criar duas chaves iguais.

**NA MESA** Cada grupo monta uma ficha de produto com etiquetas “código”, “nome”, “preço” e “estoque”. O professor solicita “o valor da chave estoque”, não “o terceiro elemento”. A atividade evidencia a diferença entre procurar pela posição e procurar por um identificador.

## 5. Consultar com segurança: colchetes, get e pertencimento

Os colchetes são adequados quando temos certeza de que uma chave existe. Se a chave não for encontrada, Python produz `KeyError`. A operação `get()` permite informar um valor alternativo e continuar a execução sem esse erro.

```
produto = {"nome": "Sabonete", "preco": 18.50}
print(produto["preco"])         # 18.5
print(produto.get("estoque"))    # None
print(produto.get("estoque", 0)) # 0
print("nome" in produto)        # True
```

A expressão `"nome" in produto` procura uma **chave**, não um valor. Já `"Sabonete" in produto.values()` verifica se o texto está entre os valores. É importante distinguir essas duas pesquisas.

| Forma de consulta | Comportamento |
|---|---|
| produto["estoque"] | Exige a existência da chave; pode gerar KeyError. |
| produto.get("estoque") | Devolve None se não existir. |
| produto.get("estoque", 0) | Devolve 0 como alternativa. |
| "estoque" in produto | Verifica a presença da chave. |

## 6. Atualizar, acrescentar e remover campos

Um dicionário é mutável. Podemos alterar o valor de uma chave já existente ou acrescentar um novo campo usando a mesma forma de atribuição. A diferença depende da existência anterior da chave.

```
produto = {"nome": "Batom", "estoque": 12}
```

```
produto["estoque"] = 9        # atualiza
produto["categoria"] = "Maquiagem"  # acrescenta
retirado = produto.pop("categoria")
print(retirado)               # Maquiagem
print(produto)
```

O método `pop()` remove a chave e devolve o valor associado. `del produto["estoque"]` remove a entrada sem devolvê-la. Para não produzir erro ao tentar excluir uma chave ausente, podemos usar `pop("chave", valor_padrao)`.

```
produto = {"nome": "Batom", "estoque": 9}
retorno = produto.pop("desconto", None)
print(retorno)  # None
print(produto)  # permanece igual
```

**DISTINÇÃO IMPORTANTE** Alterar um dicionário que está na memória **não altera automaticamente** um arquivo JSON ou um banco de dados. Para a modificação persistir no arquivo, será necessário gravar os dados de volta; esse procedimento será estudado no Caderno 03.

## 7. Percorrer chaves, valores e pares

Um laço `for` percorre diretamente as chaves de um dicionário. Para ler valores ou pares completos, utilizamos `values()` e `items()`. Esses métodos não fazem uma cópia independente do cadastro: fornecem visões das informações presentes no dicionário.

```
produto = {"codigo": 101, "nome": "Shampoo", "estoque": 18}
for chave in produto:
    print(chave)
for valor in produto.values():
    print(valor)
for chave, valor in produto.items():
    print(chave, "=", valor)
```

O último laço usa desempacotamento: a cada repetição, `items()` fornece um par (chave, valor). A chave recebe o nome do campo, e a outra variável recebe seu conteúdo. Essa técnica também facilita construir relatórios de cadastros.

### Um cálculo com dados do dicionário

```
produto = {"preco": 24.90, "estoque": 18}
valor_em_estoque = produto["preco"] * produto["estoque"]
print(f"R$ {valor_em_estoque:.2f}")  # R$ 448.20
```

No cálculo acima, preço e estoque continuam armazenados em campos separados, mas podem ser usados em uma expressão. Um dicionário não é uma calculadora: ele organiza os dados que o programa poderá processar.

## 8. Dicionários com dados combinados

O valor de uma chave não precisa ser um número ou texto simples. Uma chave pode apontar para uma lista de telefones, para outro dicionário ou para outras estruturas. Isso permite representar um pedido que contém vários itens.

```
pedido = {
    "numero": 501,
    "cliente": "Ana",
    "itens": [
        {"produto": "Shampoo", "quantidade": 2},
        {"produto": "Desodorante", "quantidade": 1}
```

```
    ]
}
print(pedido["itens"][0]["produto"])  # Shampoo
```

A leitura acontece em partes: `pedido["itens"]` devolve uma lista; `[0]` seleciona o primeiro dicionário dessa lista; `["produto"]` recupera o nome. Em seguida, teremos um caderno inteiro para praticar leituras assim, com desenhos e JSON.

## 9. Conjuntos: quando repetição não deve existir

Uma loja pode registrar várias categorias de produtos, mas em um relatório talvez só nos interessem as **categorias distintas**. Se “Higiene” aparece dez vezes em uma lista, o conjunto deve mostrar essa categoria uma única vez. Em Python, usamos `set`.

```
categorias = ["Higiene", "Cabelos", "Higiene", "Cuidados"]
unicas = set(categorias)
print(len(unicas))  # 3
print("Higiene" in unicas)  # True
```

Não interprete a ordem de impressão como ordem fixa: conjuntos não oferecem índices para localizar “a primeira categoria”. `unicas[0]` causa erro. Além disso, conjuntos só aceitam elementos que possam ser usados como valores hash, como textos, números e tuplas adequadas — não listas mutáveis.

| Estrutura | Repete valores? | Acesso |
|---|---|---|
| list | Sim | Por índice |
| tuple | Sim | Por índice |
| dict | Chaves não; valores podem | Por chave |
| set | Não | Por pertencimento, sem índice |

## 10. Adicionar, remover e comparar conjuntos

No depósito, podemos reunir as categorias que existem em cada unidade. Com conjuntos, é possível descobrir quais são comuns, quais aparecem em pelo menos uma unidade e quais existem só em uma delas.

```
centro = {"Higiene", "Cabelos", "Maquiagem"}
eldorado = {"Higiene", "Cuidados", "Perfumaria"}
print(centro & eldorado)  # comum: Higiene
print(centro | eldorado)  # todas as categorias únicas
print(centro - eldorado)  # só no Centro
```

| Operação | Significado | Símbolo / método |
|---|---|---|
| União | Tudo que aparece em pelo menos um conjunto | A \| B / A.union(B) |
| Interseção | Somente elementos comuns | A & B / A.intersection(B) |
| Diferença | Itens presentes em A, mas não em B | A - B / A.difference(B) |
| Diferença simétrica | Itens em um dos conjuntos, mas não nos dois | A ^ B |

```
categorias = {"Higiene", "Cabelos"}
categorias.add("Perfumaria")
categorias.add("Higiene")  # continua apenas uma vez
categorias.discard("Inexistente")  # não gera erro
```

O método `remove("X")` gera `KeyError` se X não existir; `discard("X")` não gera esse erro. Os dois removem X quando ele está presente. Essa distinção é importante quando os dados não são inteiramente previsíveis.

## 11. Como escolher a estrutura adequada

Não existe uma estrutura universalmente melhor. A pergunta útil é qual operação o programa precisa realizar: manter ordem, alterar posições, consultar por chave ou impedir duplicatas. A escolha errada pode tornar o código confuso mesmo quando ele ainda funciona.

| Situação | Escolha inicial | Motivo |
|---|---|---|
| Lista de produtos que será reordenada | list | Os elementos têm posição e podem ser movidos. |
| Par (latitude, longitude) lido como unidade | tuple | O registro posicional não precisa ser editado em partes. |
| Ficha de produto com campos nomeados | dict | Consulta por chaves como nome e estoque. |
| Categorias que não podem se repetir | set | Unicidade e operações entre grupos. |
| Vários produtos, cada um com várias informações | list de dict | Uma lista agrupa registros identificados por chaves. |

Uma mesma aplicação normalmente utiliza várias estruturas ao mesmo tempo. Por exemplo, o cadastro de um produto pode ser um dicionário, o catálogo inteiro uma lista de dicionários e a relação de categorias distintas um conjunto criado a partir desse catálogo.

## Para guardar deste caderno

- **Tupla**: sequência ordenada e imutável; ainda permite leitura, percurso e desempacotamento.
- **Dicionário**: associação entre chaves únicas e valores; adequado para cadastros nomeados.
- **Conjunto**: elementos distintos, sem posições por índice; útil para comparar grupos.
- A escolha deve partir das operações que o problema exige, não de decorar a sintaxe.

## Referências de consulta

Python Software Foundation. The Python Tutorial: Data Structures. https://docs.python.org/3/tutorial/datastructures.html

Python Software Foundation. Biblioteca padrão: collections e json. https://docs.python.org/3/library/collections.html ; https://docs.python.org/3/library/json.html

VisuAlgo. Visualizações e explicações de algoritmos. https://visualgo.net/en

Harvard University. CS50x 2026, Week 3 — Algorithms. https://cs50.harvard.edu/x/weeks/3/

Bhargava, Aditya. Entendendo Algoritmos (Grokking Algorithms), 2ª edição. Manning Publications. Referência de abordagem visual; exemplos e explicações deste caderno são originais.